


```

+-----+
|       |
|       | v
+-----+
| 2. TEST |
+-----+

|       |
|       | v
+-----+
| 3. SECURITY |
+-----+

|       |
|       | v
+-----+
| 4. ARTIFACT | <-- Versioned image tagged in IBM Container Registry
+-----+

|       |
|       | v
+-----+
| 5a. DEV | AUTO --> Deployed immediately
+-----+

|
| [Manual Gate: DevOps Lead + QA Lead approval]
|
| v
+-----+
| 5b. STAGING | MANUAL --> Deployed after approval
+-----+

|
| [Manual Gate: DevOps Lead + Security Director + VP E-Commerce]
|
| v
+-----+
| 5c. PRODUCTION | MANUAL --> Rolling update, zero downtime
+-----+

```

2. Pipeline Stage Configuration

The following subsections document the configuration of each stage, including its inputs, actions, outputs, and failure conditions.

Stage 1: Build

Purpose	Compile the application, build a Docker image, and push it to IBM Container Registry with a versioned tag.
Inputs	Application source code, Dockerfile, requirements.txt, Git commit SHA, IBM_API_KEY secret
Key Commands	<pre>docker build -t retail-product-service:v1.0.0-a3f9c12 . docker push us.icr.io/global-retail-platform/product-microservice:v1.0.0-a3f9c12</pre>
Outputs	Docker image in IBM ICR tagged with semantic version + commit SHA; build.env metadata file
Failure Causes	Docker build errors (missing base image, syntax errors in Dockerfile); ICR authentication failure; network timeout during push

Failure Effect	Pipeline halted at Stage 1. No test, security, or deployment activity occurs. Developer receives notification.
-----------------------	--

Stage 2: Test

Purpose	Execute unit and integration tests to verify application correctness before any deployment occurs.
Inputs	Application source code, test suite (tests/unit/, tests/integration/), requirements including pytest
Key Commands	<pre>pytest tests/unit/ --junitxml=test-results/unit-results.xml -v pytest tests/integration/ --cov=app --cov-fail-under=80</pre>
Outputs	JUnit XML test result files; coverage.xml report; pass/fail exit code
Failure Causes	Any test assertion failure; code coverage below 80% threshold; import errors or missing test dependencies
Failure Effect	Pipeline halted. The Docker image from Stage 1 is retained in the registry but never promoted. Faulty code cannot reach any environment.

Stage 3: Security Validation

Purpose	Run three independent security controls: dependency vulnerability scan, container image scan via IBM Vulnerability Advisor, and static code analysis via SonarQube.
Inputs	requirements.txt; Docker image in ICR; application source code; SONARQUBE_TOKEN secret
Scans Run	1. Safety — Python dependency CVE check 2. IBM Vulnerability Advisor — OS/package CVEs in container image 3. SonarQube — static code analysis with Quality Gate
Outputs	safety-report.json; va-report.json; SonarQube Quality Gate result; pass/fail exit code
Failure Causes	Any HIGH or CRITICAL severity CVE in dependencies or image layers; SonarQube Quality Gate failure (e.g., new vulnerabilities or code coverage drop)
Failure Effect	Image is never tagged as a release artifact. It cannot be deployed to any environment. Finding is logged for developer remediation. This protects both the platform and regulatory compliance obligations.

Stage 4: Artifact Creation

Purpose	Promote the security-verified image to a stable semantic version tag in IBM Container Registry, and generate a Software Bill of Materials (SBOM) for audit traceability.
Inputs	Docker image tagged with commit SHA (from Stage 1); confirmation that Stages 2 and 3 passed

Key Commands	<code>ibmcloud cr image-tag ...product-microservice:v1.0.0-a3f9c12 ...product-microservice:v1.0.0 cyclonedx-py --format json --output sbom.json -r requirements.txt</code>
Outputs	Stable release image tag (v1.0.0) in IBM ICR; sbom.json cataloguing all 18 software components
Failure Causes	ICR tagging errors; SBOM generation failure; network or authentication issues
Failure Effect	Without a stable artifact tag, promotion stages cannot reference a canonical image. Deployment is blocked.

Stage 5a: Deploy: Development

Purpose	Automatically deploy the verified release image to the IBM IKS development cluster and run a smoke test to confirm the service is reachable.
Inputs	Release image tag from Stage 4; IBM_API_KEY; IKS development cluster config
Gate	None — automatic promotion. Intended to provide immediate developer feedback after every successful build.
Key Commands	<code>kubectl set image deployment/product-microservice product-microservice=...v1.0.0 -n retail-dev kubectl rollout status deployment/product-microservice --timeout=120s</code>
Smoke Test	HTTP GET /products against the deployed service endpoint. Must return HTTP 200 OK.
Failure Causes	Kubernetes image pull error; rollout timeout (replicas not ready within 120s); smoke test returns non-200 status
Failure Effect	Automatic rollback triggered. Staging and Production deployments are blocked. Developer is notified.

Stage 5b: Deploy: Staging

Purpose	Promote the release image to the staging cluster for integration testing and stakeholder validation, gated by a manual approval step.
Inputs	Release image tag; IBM_API_KEY; IKS staging cluster config; manual approval from DevOps Lead and QA Lead
Gate	Manual approval required from: DevOps Engineering Lead AND QA Lead. 48-hour approval window before pipeline expires.
Outputs	Running deployment on retail-staging-cluster in the retail-staging namespace; smoke test result
Failure Causes	Approval timeout (48 hours); deployment rollout failure; smoke test failure in staging environment
Failure Effect	Production deployment remains blocked. Issue is investigated in staging without customer impact.

Stage 5c: Deploy: Production

Purpose	Deploy the validated release to the production IBM IKS cluster using a rolling update strategy, ensuring zero downtime. Notify stakeholders on success.
Inputs	Release image tag; IBM_API_KEY; IKS production cluster config; three manual approvals
Gate	Manual approval required from: DevOps Engineering Lead, Security and Compliance Director, AND VP E-Commerce. 72-hour window.
Strategy	Rolling update: maxUnavailable=0, maxSurge=1. New replica added before old one removed. Maintains full capacity throughout.
Rollback	If the post-deployment health check returns non-200, kubectl rollout undo is triggered automatically, reverting to the previous image.
Outputs	Updated production deployment; Slack notification to stakeholders with version, commit SHA, and timestamp
Failure Causes	Approval not granted within 72 hours; rollout timeout (300s); health check failure triggers automatic rollback
Failure Effect	Automatic rollback restores last known-good state. No customer-visible outage. Incident is logged.

3. Pipeline Walkthrough

The following is a simulated walkthrough of a complete pipeline execution for the product microservice, from a code push on the main branch through to production deployment. All log output is representative of actual IBM Cloud Toolchain execution.

Stage 1 — Build

```
[BUILD] Image will be tagged: us.icr.io/global-retail-platform/product-
microservice:v1.0.0-a3f9c12
[BUILD] IBM Cloud Container Registry login successful
[BUILD] Building Docker image...
  Step 1/7 : FROM python:3.12-slim
  Step 2/7 : WORKDIR /app
  Step 3/7 : COPY requirements.txt .
  Step 4/7 : RUN pip install --no-cache-dir -r requirements.txt
  Step 5/7 : COPY . .
  Step 6/7 : EXPOSE 8000
  Step 7/7 : CMD ["uvicorn", "main:app", "--host", "0.0.0.0", "--port", "8000"]
Successfully built d84f3c1a9b02
Successfully tagged product-microservice:v1.0.0-a3f9c12
[BUILD] Pushing image to IBM Container Registry...
[BUILD] Image pushed successfully
Duration: 00:02:14
STAGE RESULT: PASSED
```

Stage 2 — Test

```
[TEST] Installing dependencies...
[TEST] Running unit tests...
  tests/unit/test_products.py::test_create_product PASSED
  tests/unit/test_products.py::test_get_all_products PASSED
```

```
tests/unit/test_products.py::test_get_product_by_id PASSED
tests/unit/test_products.py::test_update_product PASSED
tests/unit/test_products.py::test_delete_product PASSED
tests/unit/test_products.py::test_missing_fields_returns_400 PASSED
tests/unit/test_products.py::test_product_not_found_returns_404 PASSED
===== 7 passed in 1.23s =====
[TEST] Running integration tests...
tests/integration/test_api_routes.py::test_full_crud_lifecycle PASSED
tests/integration/test_api_routes.py::test_partial_update_merges PASSED
===== 2 passed in 0.87s =====
[TEST] Code coverage: 91% (threshold: 80%) OK
Duration: 00:01:07
STAGE RESULT: PASSED
```

Stage 3 — Security Validation

```
[SECURITY] Scanning Python dependencies with Safety...
  Scanned 18 packages — 0 high/critical vulnerabilities found
[SECURITY] Dependency scan PASSED
[SECURITY] Scanning container image with IBM Vulnerability Advisor...
  Image: us.icr.io/global-retail-platform/product-microservice:v1.0.0-a3f9c12
  Critical: 0 | High: 0 | Medium: 1 | Low: 2
  (Medium/Low issues logged but do not block pipeline)
[SECURITY] Container image scan PASSED
[SECURITY] Running static code analysis with SonarQube...
  Quality Gate: OK
  Bugs: 0 | Vulnerabilities: 0 | Code Smells: 3 | Coverage: 91%
[SECURITY] Static analysis PASSED
Duration: 00:03:45
STAGE RESULT: PASSED
```

Stage 4 — Artifact Creation

```
[ARTIFACT] Tagging verified image as stable release...
  Source: us.icr.io/global-retail-platform/product-microservice:v1.0.0-a3f9c12
  Release: us.icr.io/global-retail-platform/product-microservice:v1.0.0
[ARTIFACT] Release image registered in IBM Container Registry
[ARTIFACT] Generating Software Bill of Materials...
  SBOM generated: sbom.json (18 components catalogued)
Duration: 00:00:38
STAGE RESULT: PASSED
```

Stage 5a — Deploy: Development (Automatic)

```
[DEPLOY:DEV] Configuring IBM IKS cluster: retail-dev-cluster
[DEPLOY:DEV] Updating deployment image to: product-microservice:v1.0.0
  deployment.apps/product-microservice image updated
  Waiting for rollout... 1/2 replicas updated... 2/2 ready
[DEPLOY:DEV] Rollout complete. Running smoke test...
  GET https://product-svc.retail-dev.ibm.com/products -> HTTP 200 OK
[DEPLOY:DEV] Smoke test PASSED
Duration: 00:01:22
STAGE RESULT: PASSED
```

Stage 5b — Deploy: Staging (Manual Gate)

```
[GATE] Approval requested: DevOps Engineering Lead, QA Lead
  APPROVED by: devops-lead (2026-03-31T09:14:22Z)
  APPROVED by: qa-lead (2026-03-31T09:21:07Z)
```

```
[DEPLOY:STG] Configuring IBM IKS cluster: retail-staging-cluster
[DEPLOY:STG] Updating deployment image to: product-microservice:v1.0.0
  deployment.apps/product-microservice image updated
  Waiting for rollout... 3/3 replicas ready
[DEPLOY:STG] Smoke test -> HTTP 200 OK  PASSED
Duration: 00:02:10  (excluding approval wait time)
STAGE RESULT: PASSED
```

Stage 5c — Deploy: Production (Manual Gate, Rolling Update)

```
[GATE] Approval requested: DevOps Lead, Security Director, VP E-Commerce
  APPROVED by: devops-lead (2026-03-31T11:02:10Z)
  APPROVED by: security-compliance-director (2026-03-31T11:08:45Z)
  APPROVED by: vp-ecommerce (2026-03-31T11:15:33Z)
[DEPLOY:PROD] Initiating rolling update (maxUnavailable=0, maxSurge=1)...
  Replica 1: new pod scheduled -> Running -> Ready (old pod removed)
  Replica 2: new pod scheduled -> Running -> Ready (old pod removed)
  Replica 3: new pod scheduled -> Running -> Ready (old pod removed)
  Replica 4: new pod scheduled -> Running -> Ready (old pod removed)
  All 4/4 replicas updated. Zero downtime achieved.
[DEPLOY:PROD] Health check -> HTTP 200 OK  PASSED
[DEPLOY:PROD] Active image: product-microservice:v1.0.0
[DEPLOY:PROD] Stakeholder notification sent via Slack
Duration: 00:03:01
STAGE RESULT: PASSED
```

Pipeline Execution Summary

Stage	Result	Duration	Key Output	Gate Type
1 — Build	PASSED	00:02:14	Docker image in IBM ICR	Automatic
2 — Test	PASSED	00:01:07	9/9 tests pass, 91% coverage	Automatic
3 — Security	PASSED	00:03:45	0 critical vulns, QG: OK	Automatic
4 — Artifact	PASSED	00:00:38	Release tag v1.0.0 + SBOM	Automatic
5a — Dev	PASSED	00:01:22	Live on dev cluster, smoke OK	Automatic
5b — Staging	PASSED	00:02:10	Live on staging, 2 approvals	Manual (2)
5c — Production	PASSED	00:03:01	Rolling update, 4 replicas live	Manual (3)
Total	SUCCESS	00:14:17	product-microservice:v1.0.0 in PROD	—

4. Reflection on Pipeline Design

4.1 How CI/CD Reduces Operational Risk

A CI/CD pipeline replaces manual, error-prone deployment processes with an automated, deterministic sequence of validated steps. In the context of the global retail platform, where a failed deployment during Black Friday could cause direct revenue loss — a key concern raised by the VP of E-Commerce — this automation fundamentally changes the risk profile of every release. Each stage acts as a gate: if a build fails, tests never run; if tests fail, the image never enters the registry; if security scans detect critical vulnerabilities, no deployment is attempted. This means that only code that has survived every automated check can reach any environment, which dramatically reduces the chance of defective or insecure software reaching production. The pipeline also produces a complete audit trail — build logs, test reports, security scan results, approval records, and deployment timestamps — that makes every release fully traceable and reproducible.

4.2 Why Security Scanning Must Be Part of the Pipeline

Embedding security controls directly into the pipeline — rather than treating them as a separate, post-deployment activity — enforces security as a non-negotiable condition of release rather than an optional review. The Security and Compliance Director identified hybrid cloud migration as a risk surface for exposing customer and transaction data if controls are not enforced from the outset. By placing three independent scans (dependency CVE check with Safety, container image analysis with IBM Vulnerability Advisor, and static code analysis with SonarQube) before the artifact creation stage, the pipeline ensures that no image with a known high-severity vulnerability can ever be tagged as a release artifact or deployed to any environment. This shift-left approach catches vulnerabilities when they are cheapest to fix — during development — rather than after they have been promoted to production infrastructure. It also supports continuous compliance with GDPR and PDPA obligations by providing machine-readable scan evidence that can be submitted to auditors.

4.3 How Promotion Gates Support Compliance Requirements

The three-tier promotion model — automatic deployment to Development, manual approval for Staging, and three-role approval for Production — directly maps to the compliance and governance requirements identified in the stakeholder register. The Security and Compliance Director's requirement for enforced access controls is satisfied by requiring the Director's explicit sign-off before any code reaches the production environment. The DevOps Engineering Lead's objective of eliminating manual intervention risk from release processes is addressed by making the lower environments fully automated, while preserving human oversight at the stages where business impact is highest. The VP of E-Commerce's concern about platform stability during peak events is addressed by the rolling update strategy in the Production stage, which maintains full replica capacity throughout the rollout and triggers an automatic rollback if the post-deployment health check fails. Taken together, these gates ensure that every production deployment is deliberate, traceable, and reversible — all of which are foundational requirements for operating a regulated, multi-region hybrid cloud platform.